

Clase 16

Clase resumen de Python

Lógica booleana

and

V	V	V
V	F	F
F	V	F
F	F	F

or

V	V	V
V	F	V
F	V	V
F	F	F

not

V	F
F	V



Condicionales

if prop1 **or** prop2:

 print('comprobe solo if y termine')

elif prop1 **and** pro2:

 print('comprobe if y elif y termine')

elif not prop3:

 print('comprobe if, elif1, elif2 y termine')

else:

 print('comprobe if, elif1, elif2 e hice else y termine')



Mutable vs. Inmutable

Tipo	Mutable/Immutable
numérico	Immutable
booleano	Immutable
cadena	Immutable
tupla	Immutable
lista	Mutable
diccionario	Mutable
conjunto	Mutable

Listas vs. Tuplas vs. Diccionarios vs. Conjuntos

Listas	Tuplas	Diccionarios	Conjuntos
• Mutables • Ordenado • Indexación • [] • list()	• Inmutables • Ordenado • Indexación • (,) • tuple()	• Mutables • No ordenado • {Clave : Valor} • Claves inmutables • Claves únicas	• Mutables • No ordenado • Elementos inmutables • Elementos no duplicados

Métodos de cadenas

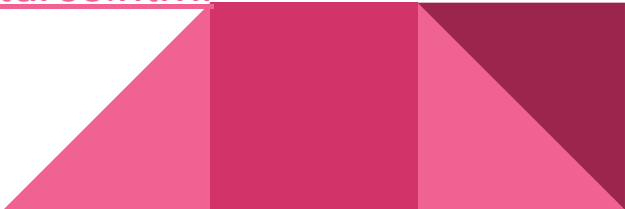
`cadena = "Python es lo más!"`

- `cadena.upper()` # "PYTHON ES LO MÁS!"
 - `cadena.lower()` # "python es lo más!"
 - `cadena.islower()` # False
 - `cadena.isupper()` # False
 - `cadena.title()` # "Python Es Lo Más!"
 - `cadena.count("s")` # 2
 - `cadena.split(" ")` # ["Python", "es", "lo", "más!"]
- 

Algunas funciones de listas

- `lista.copy()` # Devuelve una copia en otra con otro id
- `lista.index(elem)` # Retorna el índice de elem
- `lista.append(elem)` # Agrega elem al final de la lista
- `lista.remove(elem)` # Quita el primer ítem de la lista cuyo valor sea elem
- `lista.count(elem)` # Retorna la cantidad de veces que elem aparece en la lista
- `lista.sort()` # Ordena los elementos de la lista
- `lista.pop(i)` # Quita y devuelve el elemento de la posición i

Más info <https://docs.python.org/es/3/tutorial/datastructures.html>



Acceso a un elemento

```
lista = [True, (3, 2, 1), 'Hola', 12]
```

```
print(lista[1][2]) # 1
```

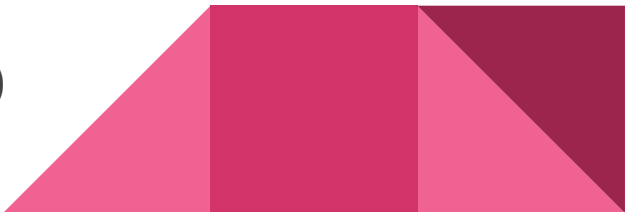
```
lista.append([( 'a', 'b'), ( 'c', 'd')]) # [True, (3, 2, 1), 'Hola', 12, [( 'a', 'b'), ( 'c', 'd')]]
```

```
print(lista[4][1][0]) # 'c'
```

```
tupla = tuple(lista) # (True, (3, 2, 1), 'Hola', 12, [( 'a', 'b'), ( 'c', 'd')])
```

```
tupla[4][1] = 'CAMBIO'
```

```
print(tupla) # (True, (3, 2, 1), 'Hola', 12, [( 'a', 'b'), 'CAMBIO'])
```



Recorrer una lista

```
for elem in lista:
```

```
    print(elem)
```

```
for indice in range(len(lista)):
```

```
    print(lista[indice])
```

```
lista = [1, 2, 3]
```

```
i = 0
```

```
while True:
```

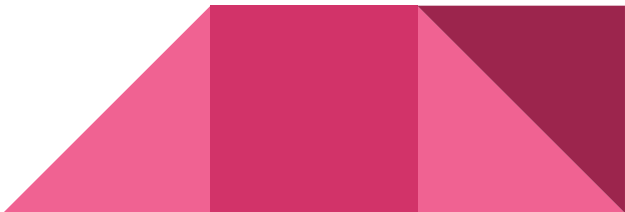
```
    try:
```

```
        print(lista[i])
```

```
        i += 1
```

```
    except:
```

```
        break
```



Diccionarios

```
dicci = {'alumno': 'Juan',
```

```
        'notas': [4, 8, 10, 5]}
```

```
dicci['nacimiento'] = '12/06/98'
```

```
print(dicci) # {'alumno': 'Juan', 'notas': [4, 8, 10, 5], 'nacimiento': '12/06/98'}
```

```
dicci['alumno'] = 'Juan Castro'
```

```
print(dicci) # {'alumno': 'Juan Castro', 'notas': [4, 8, 10, 5], 'nacimiento': '12/06/98'}
```



Recorrer diccionarios

```
musica = {'rock': ['the beatles', 'radiohead', 'rata blanca'],  
         'punk': ['ramones', 'attaque 77']}
```

#las claves

```
for elem in musica:
```

```
    print(elem)
```

los valores

```
for elem in musica:
```

```
    print(musica[elem])
```

los valores

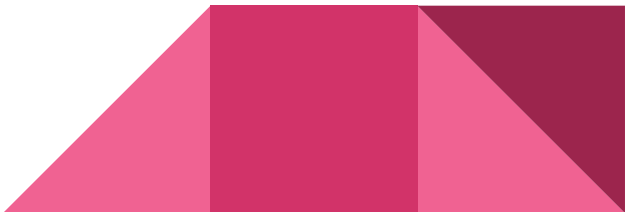
```
for elem in musica.values():
```

```
    print(elem)
```

tuplas

```
for elem in musica.items():
```

```
    print(elem)
```



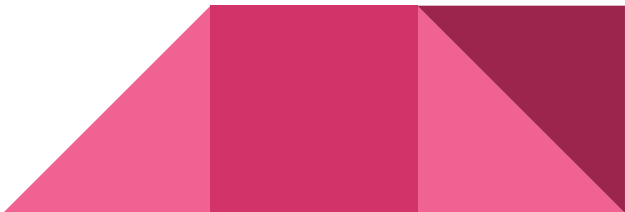
Algunos Métodos

- `dicci.keys()` # Devuelve una lista con las claves
 - `dicci.values()` # Devuelve una lista con los valores
 - `dicci.items()` # Devuelve una lista de tuplas (key, value)
 - `dicci.pop(key)` # Elimina el elemento y devuelve la valor de la key
 - `del dicci[key]` # Elimina el elemento con clave key
 - `dicci.popitem()` # Elimina y devuelve el último elemento insertado
 - `dicci.update(iterable)` # Inserta los elementos del iterable clave: valor
 - `dict.fromkeys(keys, value)` # Devuelve un diccionario con el mismo value para todas las keys
- 

Cantidad de parámetros indeterminado (*args, **kwargs)

```
def funcion(a, b, *args, **kwargs):  
    print(a, b, end=' ')  
    for elem in args:  
        print(elem, end=' ')  
    for key in kwargs:  
        print(key, kwargs[key], end=' ')  
    print('')
```

```
funcion(1, 2, 3, 4, 5, seis=6, siete=7) # 1 2 3 4 5 seis 6 siete 7  
funcion(1, 2)                          # 1 2  
funcion(1, 2, 3, 4, 5)                  # 1 2 3 4 5  
funcion(1, 2, seis=6, siete=7)          # 1 2 seis 6 siete 7
```



Anotaciones de tipo

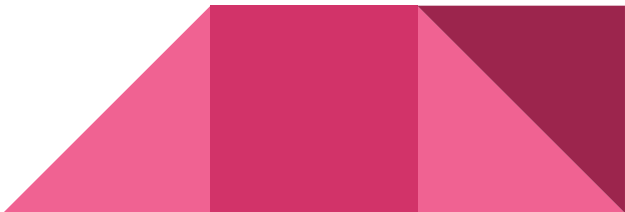
En Python se puede sugerir el tipo de cada parámetro que recibe una función, así como también el tipo del valor de retorno.

```
def saludar(nombre: str,  apellido: str = "González") -> None:
```

```
    print(f"Hola, {nombre} {apellido}!")
```

```
def calculo(num1: int,  num2: float = 1.0,  tup = (0, 0, 0)) -> float:
```

```
    return num1/num2 + tup[2]
```



Funciones anidadas, global y nonlocal

global y nonlocal permiten acceder a variables no locales a la función.

```
x = 0
y = 2
def uno():
    x = 1
    y = 22
    def uno_uno():
        # nonlocal x
        # global x
        x = 11
        print(f"En uno_uno: {x} -- {y}")
    uno_uno()
    print(f"En uno: {x} -- {y}")
uno()
print(f"En ppal: {x} -- {y}")
```



lambda, map(), filter(), sorted()

```
lista = [1, 2, 3, 4, 5, 6, 7]
```

```
dobles2 = list(map(lambda x: x*2, lista))
```

```
pares2 = list(filter(lambda x: x%2==0, lista))
```

```
lista = [(4, 1), (-6, 12), (10, 2)]
```

```
lista_ord= sorted(lista, key=lambda x: x[1])
```



Estructura de manejo de excepciones

try:

Lo que pueda ocasionar una excepcion
<codigo>

except:

Debe haber una o mas

Se ejecuta si ocurre una excepción en try
<codigo>

else:

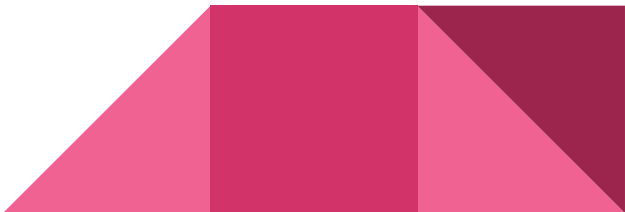
Opcional

Se ejecuta si no ocurre ninguna excepcion en try
<codigo>

finally:

Opcional

Siempre se ejecuta
<codigo>



Archivos en Python

Texto	Json	Pickle
• Solo maneja strings. • No hay necesidad de importar ningún módulo. • Brinda más funcionalidades por defecto.	• Solo maneja objetos compatibles con JavaScript • Es la notación más usada en la web. • Es compatible con muchos sistemas.	• Maneja cualquier tipo de objeto. • Los archivos creados solo pueden ser deserializados por Python. • Se debe confiar en el contenido del archivo.



Ejercicio integrador

Escribe un programa que permita a un usuario gestionar una lista de contactos. El programa debe ser capaz de realizar las siguientes operaciones:

1. Agregar un nuevo contacto: El usuario puede ingresar un nombre y un número de teléfono para agregar un nuevo contacto a la lista.
2. Buscar un contacto: El usuario puede buscar un contacto por su nombre y mostrar su número de teléfono.
3. Listar todos los contactos: El usuario puede ver la lista completa de contactos.
4. Guardar la lista de contactos en un archivo JSON: El programa debe permitir al usuario guardar la lista actual de contactos en un archivo JSON.
5. Cargar la lista de contactos desde un archivo JSON: El usuario debe poder cargar una lista de contactos previamente guardada desde un archivo JSON.
6. Salir del programa: El usuario puede salir del programa en cualquier momento.